Assembly

©2022 All rights reserved

Introduction

The Goal

Get you able to read assembly

Focus on ARM64

Then ARM32 and x86-64 differ

Some talking, some exercises, repeat

Exercises

Reading and Writing

Regular and Bonus questions

Follow setup instructions

ARM64 - Basics

What Is It?

ARM64, AArch64, ARMv8-A

64-bit

32-bit instructions

RISC

https://developer.arm.com/documentation/ddi0602/2025-06

Instruction Layout

```
add x0, x1, x2
```

Opcode Operands

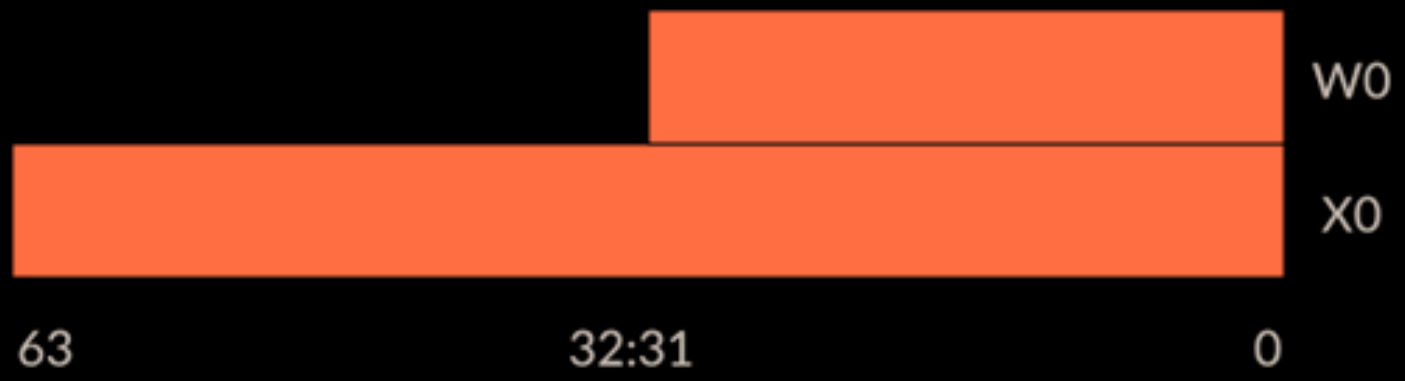
```
x0 = x1 + x2
```

General-Purpose Registers

Small, fast memory locations that can be operated on by instructions

Numbered 0-30

64-bit (x) 32-bit (w)



Arithmetic Instructions

add

sub

mul

udiv sdiv

Signedness

Signedness depends on the instruction

```
x0 = 0xfffffffffffffffd
18446744073709551613 -3
```

Immediates

Constant value

Prefixed with #

```
add x0, x0, #1
```

Not all instructions accept it

Limited in their value

Zero Register

`xzr`

Can be used wherever a general-purpose register can

```
mul x0, x1, x2 = madd x0, x1, x2, xzr
```

Exercises

Bitwise Operations

```
mvn x0, x1
x1 = 0b1010
-----------
x0 = 0b0101
```

```
orr x0, x1, x2
x1 = 0b1010
x2 = 0b1100
-----------
x0 = 0b1110
```

```
and x0, x1, x2
x1 = 0b1010
x2 = 0b1100
-----------
x0 = 0b1000
```

```
eor x0, x1, x2
x1 = 0b1010
x2 = 0b1100
-----------
x0 = 0b0110
```

Shifting Operations

```
lsl w0, w1, #4
w1 = 0xfebcda98
---------------
w0 = 0xebcda980
```

0b11111100

-4

```
lsr w0, w1, #4
w1 = 0xfebcda98
---------------
w0 = 0x0febcda9
```

0b01111110

126

```
asr w0, w1, #4
w1 = 0xfebcda98
---------------
w0 = 0xffebcda9
```

```
ror w0, w1, #4
w1 = 0xfebcda98
---------------
w0 = 0x8febcda9
```

0b11111110

-2

Moving Things Around

```
mov <xd>, <xm> = orr <xd>, zxr, <xm>
movz <xd>, #<imm>{, lsl #<shift>}
movk <xd>, #<imm>{, lsl #<shift>}
133337 = 0x208d9
movz x0, #0x08d9
movk x0, #0x2, lsl #16
```

Exercises

ARM64 - Control Flow

An Infinite Loop

```
foo:
    b foo
```

Program Counter

pc

Points to the current instruction

Comparison and Conditional Branches

```
cmp <xn>, <xm>
```

```
b.eq - ==        b.lo - u<        b.lt - s<
b.ne - !=        b.ls - u<=       b.le - s<=
                 b.hi - u>        b.gt - s>
                 b.hs - u>=       b.ge - s>=
```

```
cbz/cbnz <xt>, <label>
tbz/tbnz <xt>, #<imm>, <label>
```

Examples

```
  cmp x0, x1
  b.hi x0_higher
x1_higher:
  mov x2, x1
  b end
x0_higher:
  mov x2, x0
end:
```

```
if x0 > x1:
  x2 = x0
else:
  x2 = x1
```

```
  movz x1, #1
loop:
  cbz x0, end
  mul x1, x1, x0
  sub x0, x0, #1
  b loop
end:
```

```
x1 = 1
while x0 != 0:
  x1 *= x0
  x0 -= 1
```

What's Really Going On?

```
cmp <xn>, <xm> = subs xzr, <xn>, <xm>
```

```
n - Negative
z - Zero
c - Carry (Unsigned)
v - Overflow (Signed)
```


INTERRUPT LABS

Subtraction in Terms of Addition

```
a - b = a + -b

-8 4 2 1
 0 0 1 1 = 3
 1 1 0 0 = -4
 1 1 0 1 = -3

-b = ~b + 1
a - b = a + ~b + 1

Carry = a >= b
```

Conditions and Flags

```
eq - z=1
ne - z=0

lo - c=0
ls - z=1 & c=0
hi - z=0 & c=1
hs - c=1

lt - n!=v
le - z=0 & n!=v
gt - z=0 & n=v
ge - n=v
```

Exercises

Flag Setting Instructions

```
cmn <xn>, <xm> = adds xzr, <xn>, <xm>
tst <xn>, <xm> = ands xzr, <xn>, <xm>
ccmp/ccmn <xn>, <xm>, #<nzcv>, <cond>
        adcs/sbcs <xd>, <xn>, <xm>
```

Examples

```
adds x0, x10, x20
adcs x1, x11, x21
adcs x2, x12, x22
```

```
x2:x1:x0 = x12:x11:x10 +
           x22:x21:x20
```

```
cmp x0, x1
ccmp x1, x2, #0b0010, lo
b.lo branch
```

```
x0 < x1 < x2
```

Avoiding Branches

```
csel <xd>, <xn>, <xm>, <cond>
csinc <xd>, <xn>, <xm>, <cond>
csinv <xd>, <xn>, <xm>, <cond>
csneg <xd>, <xn>, <xm>, <cond>
```

Functions (Kinda)

```
bl add_function
...

add_function:
  add x0, x0, x1
  ret
```

x30 = lr

Exercises

ARM64 – Memory and Stack

Loading and Storing

`ldr <xt>, ???`	`str <xt>, ???`
`ldp <xt1>, <xt2>, ???`	`stp <xt1>, <xt2>, ???`
`[<xn>{, #<imm>}]`	Regular
`[<xn>, #<imm>]!`	Pre-Index
`[<xn>], #<imm>`	Post-Index
`<label>`	Only ldr
`[<xn>, <xm>{, lsl #3}]`	Only ldr/str

Examples

```
0x1000: 00 11 22 33 44 55 66 77  88 99 aa bb cc dd ee ff
x0 = 0x1000
```

```
ldr x1, [x0, #8]
x1 = 0xffeeddccbbaa9988
x0 = 0x1000
```

```
ldr x1, [x0, #8]!
x1 = 0xffeeddccbbaa9988
x0 = 0x1008
```

```
ldr x1, [x0], #8
x1 = 0x7766554433221100
x0 = 0x1008
```

Size Variants

	Load (Unsigned)	Load (Signed)	Store
32-bit		ldrsw	
16-bit	ldrh	ldrsh	strh
8-bit	ldrb	ldrsb	strb

Getting Addresses Into Registers

```
adr <xd>, <label>
adrp <xd>, <label>
```

Exercises

Stack

sp points to the top

Full-descending

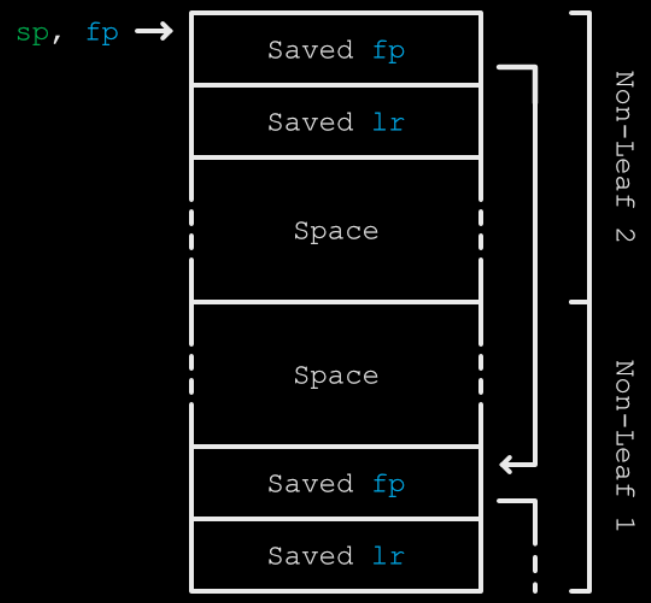
16-byte aligned

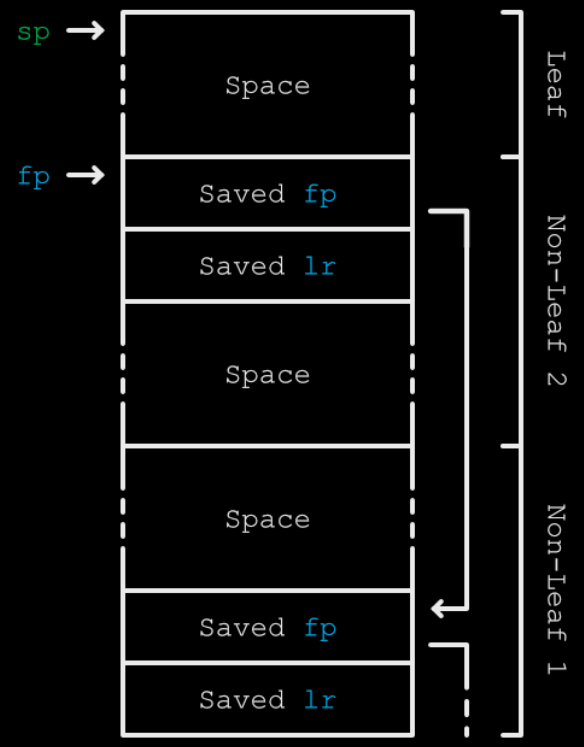
In some instructions, sp replaces xzr

```
Push: stp <xt1>, <xt2>, [sp, #-16]!
 Pop: ldp <xt1>, <xt2>, [sp], #16
```

```
0x7fc0: 0x8888888888888888 <- sp
0x7fc8: 0x7777777777777777
0x7fd0: 0x6666666666666666
0x7fd8: 0x5555555555555555
0x7fe0: 0x4444444444444444
0x7fe8: 0x3333333333333333
0x7ff0: 0x2222222222222222
0x7ff8: 0x1111111111111111
```

Call Stack






```
sub sp, sp, #32
stp fp, lr, [sp, #16]
add fp, sp, #16

ldp fp, lr, [sp, #16]
add sp, sp, #32
ret
```

```
stp fp, lr, [sp, #-32]!
mov fp, sp

ldp fp, lr, [sp], #32
ret
```

```
sub sp, sp, #16

add sp, sp, #16
ret
```

Calling Convention

```
Arguments: x0-x7, [sp], [sp, #8], ...
Return: x0, x1
Indirect Return: xr (x8)
Caller Saved: x9-x18
Callee Saved: x19-x28
```

Exercises

ARM64 - Miscellaneous

System Calls (Linux)

```
Number: x8
Arguments: x0-x5
svc #0
Return: x0
```

Example

```
movz x8, #63
movz x0, #0
movz x1, #0x1000
movz x2, #16
svc #0
```

```
read(STDIN_FILENO, 0x1000, 16)
```

BTI

Branch Target Identification

Indirect branches are BTI-setting

The next instruction to execute needs to be BTI-clearing

`bti {j}{c}`

`ret` is not BTI-setting

PAC

Pointer Authentication Code

Pointer, Key, Context

Signature stored in high 16 bits

`paciasp`

`retaa`

ARM32 - Basics

What Is It?

ARM32, AArch32, ARMv7-A/M, ARMv8-M

32-bit

32/16-bit instructions

RISC

https://developer.arm.com/documentation/ddi0597/2025-06

Registers

32-bit

`r0-r15`

`r11 = fp`

`r13 = sp`

`r14 = lr`

`r15 = pc`

No zero register

Two Modes

ARM and Thumb

32-bit and 16/32-bit instructions

Programs can contain a mix

Moving Immediates

```
mov <rd>, #<imm16>
movt <rd>, #<imm16>
```

What's the Same?

Instructions look the same

`add, sub, mul, udiv, sdiv`

`mvn, orr, and, eor`

`lsl, lsr, asr, ror`

`mov`

Exercises

ARM32 – Control Flow

Basic Similarities and Differences

`b, cmp, b.<cond>` (Same but no dot)

`cbz, cbnz` (Only Thumb)

Flags (Same)

`cmn, tst` (Same)

`ccmp, ccmn` (Don’t exist)

`adcs, sbcs` (Same)

`csel, csinc, csinv, csneg` (Don’t exist)

All Instructions Are Conditional

```
<opcode>{<condition>}
```

Examples

```
cmp r0, #0
moveq r1, r2
movne r1, r3
```

```
if r0 == 0:
  r1 = r2
else:
  r1 = r3
```

```
cmp r0, r1
cmplo r1, r2
blo branch
```

```
r0 < r1 < r2
```

Conditional Instructions in Thumb

```
it{t|e}{t|e}{t|e} <cond>
t - then (cond)
e - else (!cond)
```

Examples

```
cmp r0, #0
moveq r1, r2
movne r1, r3
movgt r4, r5
```

```
cmp r0, #0
addhs r1, #1
sublo r1, #2
andhs r2, r2, #0xfe
orrhs r2, r2, #1
```

```
cmp r0, #0
ite eq
moveq r1, r2
movne r1, r3
it gt
movgt r4, r5
```

```
cmp r0, #0
itett hs
addhs r1, #1
sublo r1, #2
andhs r2, r2, #0xfe
orrhs r2, r2, #1
```

Changing Mode

```
blx <label>
lr lsb = 0 (ARM) or 1 (Thumb)
bx <rm>
```

Exercises

ARM32 - Memory and Stack

Basic Similarities and Differences

`ldr/str` (Mostly same)

`ldp/stp` (Doesn’t exist)

`adr` (Same)

`adrp` (Doesn’t exist)

Load and Store Multiple

```
ldm/stm <rn>{!}, <registers>
{r1-r5, r7, r8-r10}
ldmia/stmia
ldmib/stmib
ldmda/stmda
ldmdb/stmdb
```

Stack

sp points to the top

Full-descending

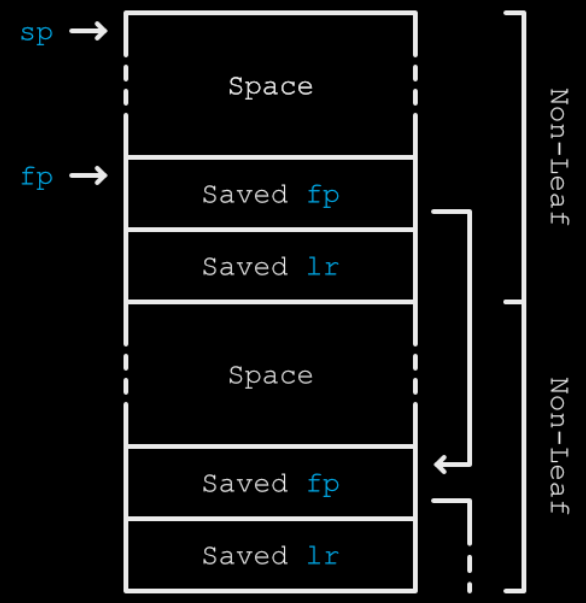
4-byte aligned

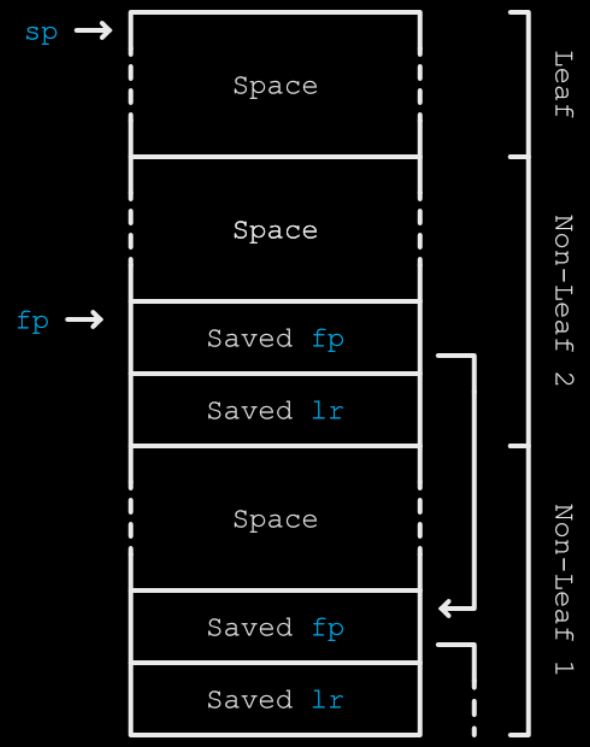
```
Push: push = stmdb sp!
 Pop: pop = ldmia sp!
```

```
0x7fe0: 0x88888888 <- sp
0x7fe4: 0x77777777
0x7fe8: 0x66666666
0x7fec: 0x55555555
0x7ff0: 0x44444444
0x7ff4: 0x33333333
0x7ff8: 0x22222222
0x7ffc: 0x11111111
```


INTERRUPT LABS

Call Stack






```
push {fp, lr}
mov fp, sp
sub sp, sp, #8

mov sp, fp
pop {fp, pc}
```

```
sub sp, sp, #8

add sp, sp, #8
bx lr
```

Calling Convention

```
Arguments: r0-r3, [sp], [sp, #4], ...
Return: r0, r1
Indirect Return: r0
Caller Saved: r12
Callee Saved: r4-r10
```

Exercises

ARM32 - Miscellaneous

System Calls (Linux)

```
Number: r7
Arguments: r0-r5
svc #0
Return: r0
```

Exist in ARMv8-M
Instructions differ

x86-64 - Basics

What Is It?

x86-64, x64, AMD64, Intel 64

64-bit

Variable length instructions

CISC

https://www.felixcloutier.com/x86/

Instruction Layout

```
add rax, rbx
```

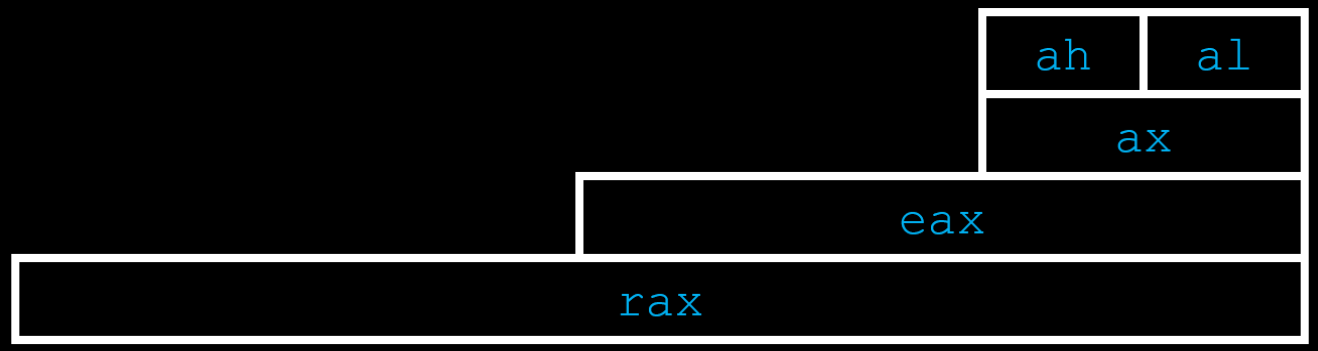
Opcode Operands

```
rax = rax + rbx
```


INTERRUPT LABS

General-Purpose Registers

64-bit	32-bit	16-bit	8-bit (high)	8-bit (low)
rax-rdx	eax-edx	ax-dx	ah-dh	al-dl
rsi	esi	si		sil
rdi	edi	di		dil
rbp	ebp	bp		bpl
rsp	esp	sp		spl
r8-r15	r8d-r15d	r8w-r15w		r8b-r15b



Operands

Often any of:

Immediate

Register

Memory Location

Arithmetic Operations

add/sub r/m, i/r/m

No mixing of sizes

Only one memory location

Maximum 32-bit immediate (signed)

mul/imul r/m

rdx:rax = rax * r/m

div/idiv r/m

rax = rdx:rax / r/m rdx = rdx:rax % r/m

inc/dec r/m

INTERRUPT LABS

Bitwise and Shifting Operations

```
not r/m
and/or/xor r/m, i/r/m
shl/shr/sar/ror r/m, i/cl
```

Moving Things Around

```
mov r/m, i/r/m
mov r, i64
```

Exercises

x86-64 - Control Flow

An Infinite Loop

```
foo:
    jmp foo
```

Instruction Pointer

`rip`

Points to the current instruction

Comparison and Conditional Jumps

```
cmp r/m, i/r/m
```

je - ==	jb - u<	jl - s<
jne - !=	jbe - u<=	jle - s<=
	ja - u>	jg - s>
	jae - u>=	jge - s>=

What's Really Going On?

```
SF - Sign (Negative)
ZF - Zero
CF - Carry (Unsigned)
OF - Overflow (Signed)
```

Most instructions set flags

Flag Setting Instructions

```
test r/m, i/r
adc/sbb r/m, i/r/m
```


INTERRUPT LABS

Avoiding Jumps

```
set<cond> r/m
cmov<cond> r, r/m
```

Exercises

x86-64 - Memory and Stack

Memory Locations

```
[base + index * scale + displacement]
base - register
index - register
scale - 1, 2, 4, 8
displacement - 32-bit immediate (signed)
```

Any parts can be removed

Specifying the Size

```
byte ptr - 8-bit
word ptr - 16-bit
dword ptr - 32-bit
qword ptr - 64-bit
```

Quick Maths

```
lea r, m
lea rax, [rax + rbx * 2 + 7]
```

Stack

`rsp` points to the top

Full-descending

16-byte aligned (relaxed)

`Push: push i/r/m`

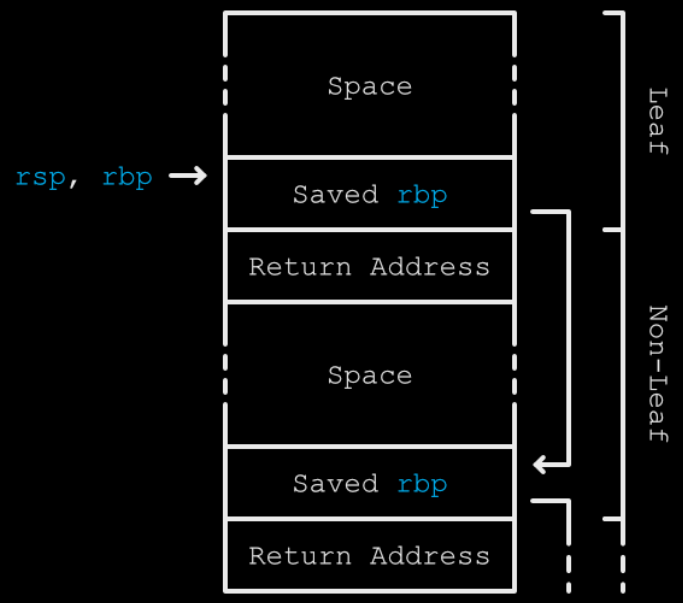
`Pop: pop r/m`

```
0x7fc0: 0x8888888888888888 <- rsp
0x7fc8: 0x7777777777777777
0x7fd0: 0x6666666666666666
0x7fd8: 0x5555555555555555
0x7fe0: 0x4444444444444444
0x7fe8: 0x3333333333333333
0x7ff0: 0x2222222222222222
0x7ff8: 0x1111111111111111
```

Calling and Returning

```
call <label>
ret
```

Call Stack




```
push rbp
mov rbp, rsp
sub rsp, 16

add rsp, 16
pop rbp
ret
```

```
push rbp
mov rbp, rsp

pop rbp
ret
```

Calling Convention

```
Arguments: rdi, rsi, rdx, rcx, r8, r9, [rsp + 8], [rsp + 16], ...
Return: rax, rdx
Indirect Return: rdi
Caller Saved: r10, r11
Callee Saved: rbx, r12-r15
```

Exercises

x86-64 - Miscellaneous

System Calls (Linux)

```
Number: rax
Arguments: rdi, rsi, rdx, r10, r9, r8
syscall
Return: rax
```

Control-flow Enforcement Technology

endbr64

Shadow Stack

INTERRUPT LABS

www.interruptlabs.co.uk